SYNTH-01: Synthetic Monitoring Fundamentals

> **Series:** SYNTH — Synthetic Monitoring | **Notebook:** 1 of 6 | **Created:** December 2025 | **Last Updated:** 10/13/2026

Understanding Proactive Availability and Performance Testing

This notebook introduces Dynatrace Synthetic Monitoring, which enables proactive testing of application availability, functionality, and performance from locations around the world.

Table of Contents

1. What is Synthetic Monitoring?
2. Monitor Types
3. Key Concepts
4. Synthetic Data in Grail
5. Your First Synthetic Query

Prerequisites

- ✅ Access to a Dynatrace environment with Synthetic Monitoring enabled
- ✅ DQL query permissions (viewer role minimum)
- ✅ Basic understanding of web applications and APIs

Sprint 1.337 (April 2026) Updates

Two sprint-1.337 changes touch synthetic monitoring:

- **Primary Grail tags on monitors** (Latest Dynatrace) — set per monitor; every event and metric the monitor produces is then enriched with `primary_tags.*`. The tags come from the **monitor**, not from the host a private-location ActiveGate runs on: *"Tags aren't inherited from the Synthetic location, host group, or any other entity."* Private locations need **ActiveGate 1.331+** for the enrichment (SYNTH-04). Useful for cost-attribution dashboards that want to bill synthetic execution by team or product line. **FAQ-26** places monitor tags and security context in an environment-wide metadata strategy.
- **Configuration API → Settings v2 + Platform tokens** for new automation. Synthetic settings are split across per-aspect Settings schemas under `builtin:synthetic.*` — for example `builtin:synthetic.http.outage-handling`, `builtin:synthetic.browser.scheduling`, `builtin:synthetic.primary-grail-tags` and `builtin:synthetic.security-context`. There is no single `builtin:synthetic_test` schema; monitor definitions themselves go through the Synthetic API. Note the v1.88.0 Terraform provider boundary still requires API tokens for synthetic monitors specifically (see AUTOM-04 § Authentication Boundary). For new automation on Latest Dynatrace, *"Dynatrace recommends migrating to the Synthetic platform permissions and the Synthetic Platform API to manage Synthetic in Latest Dynatrace"* — it authenticates with a bearer platform token.

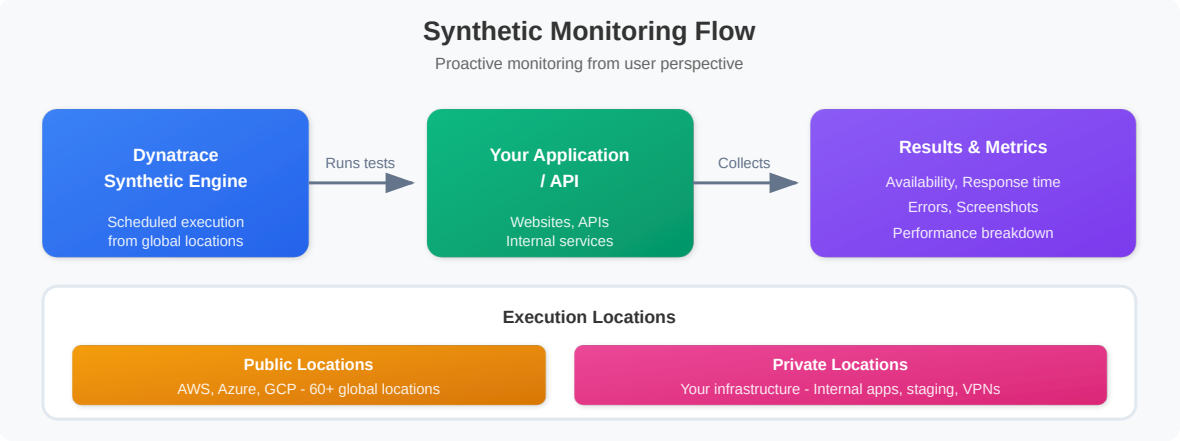
> **Sources:** Primary Grail tags for Synthetic (DT docs) — *"After that, all events and metrics generated by the monitor are enriched with these primary Grail tags."* and *"If you're using private Synthetic locations, install ActiveGate version 1.331+. Earlier versions don't enrich the metrics and events produced by Synthetic monitor executions with the monitor's primary Grail tags."*; Settings API schemas (DT docs) — 22 `builtin:synthetic.*` schema IDs listed and none named `builtin:synthetic_test`, read 09/18/2026; Access control (DT docs) — the Synthetic Platform API recommendation, read 10/02/2026.

---

1. What is Synthetic Monitoring?

Synthetic Monitoring simulates user interactions with your applications from external locations, providing:

- **Proactive Detection**: Find issues before real users encounter them
- **24/7 Availability Testing**: Monitor even when no users are active
- **Global Performance Baseline**: Measure response times from multiple locations
- **SLA Validation**: Verify service level agreements are being met
- **Third-Party Dependency Monitoring**: Test external APIs and services



Synthetic vs Real User Monitoring (RUM)

Aspect	Synthetic Monitoring	Real User Monitoring
Data Source	Simulated transactions	Actual user sessions
Coverage	24/7, consistent	Only when users active
Locations	Controlled, specific	Wherever users are
Use Case	Baseline, SLA, proactive	Actual experience
Cost	By synthetic usage (see your rate card)	Per session

2. Monitor Types

Dynatrace offers three kinds of synthetic monitor, plus ingestion of results from third-party tools:

Browser Monitors (Single-URL and Browser Clickpath)

Type	Description	Use Case
Single-URL	Load a single page	Homepage availability
Browser Clickpath	Multi-step user journey	Login flows, checkout

Capabilities:
- Full browser rendering (Chromium)
- JavaScript execution
- Content validation per step
- Duration per monitor and per step (SYNTH-02 § 5)

HTTP Monitors

Type	Description	Use Case
Single Request	One HTTP call	API health check
Multi-step	Chained requests	API workflow validation

Capabilities:
- Any HTTP method (GET, POST, PUT, DELETE)
- Custom headers and authentication
- Response validation (status code, text/regex, SSL certificate expiry)
- Lightweight and fast execution

Network Availability Monitors (NAM)

ICMP, TCP and DNS checks of hosts and services that have no HTTP endpoint. *"Network availability monitors are supported only on private Synthetic locations."* See SYNTH-05.

Third-party results

Results from external synthetic tools can be ingested through the third-party synthetic API; this series does not cover it.

> **Sources:** Network availability monitoring (DT docs) — *"Network availability monitors are supported only on private Synthetic locations."*

3. Key Concepts

Execution Locations

Location Type	Description	Best For
Public	Dynatrace-hosted worldwide	External-facing apps
Private	Your infrastructure	Internal apps, security

Execution Frequency

How often the monitor runs:
- **Browser**: every 5, 10, 15 or 30 minutes; every 1, 2 or 4 hours; or on demand only

- **HTTP**: the same options, plus a 1-minute frequency (1-minute HTTP monitors were observed on the validation tenant, 09/18/2026)

> **Sources:** Create and configure a browser monitor (DT docs) — *"You can choose a frequency of every 5 , 10 , 15 , or 30 minutes; or 1 , 2 , or 4 hours. You can also set up your monitor to be executed On demand only ."*

Outage Detection

Dynatrace detects outages based on:
- Consecutive failures from single location
- Failures from multiple locations simultaneously
- Local outage vs global outage classification

Key Metrics

Synthetic measurements are surfaced two ways in Grail (see §4): as **metric keys** queried with `timeseries`, and as fields on execution **events** queried with `fetch dt.synthetic.events`. The table maps each concept to both.

Concept	Metric key (`timeseries`)	Event field (`fetch dt.synthetic.events`)	Monitor Type
Availability (%)	`dt.synthetic.http.availability` / `dt.synthetic.browser.availability`	derived from `result.state == "SUCCESS"`	All
Total duration	`dt.synthetic.http.duration` / `dt.synthetic.browser.duration`	`result.statistics.duration`	All
DNS lookup	`dt.synthetic.http.dns`	`result.statistics.host_name_resolution_time`	HTTP
TCP connect	`dt.synthetic.http.tcp_connect_time`	`result.statistics.tcp_connect_time`	HTTP
TLS handshake	`dt.synthetic.http.tls_handshake_time`	`result.statistics.tls_handshake_time`	HTTP
Time to first byte	`dt.synthetic.http.time_to_first_byte`	`result.statistics.time_to_first_byte`	HTTP
Executions (count)	`dt.synthetic.http.executions` / `dt.synthetic.browser.executions`	`count()` over the events	All

> **Note on units:** the `dt.synthetic.*.duration` *metric* is already expressed in milliseconds. The `result.statistics.*` *event* fields are `duration`-typed (nanoseconds) — divide by `1ms` to get milliseconds, e.g. `result.statistics.duration / 1ms`.

4. Synthetic Data in Grail

With **Synthetic on Grail**, execution results are stored as **synthetic events** in Grail and are also rolled up into **synthetic metrics**. There are two complementary query paths:

Path 1 — Execution events (`fetch dt.synthetic.events`)

One record per execution (and per step). This is where per-failure diagnostics, status messages, SSL certificate data, and per-step timings live.

Data object	Description
`dt.synthetic.events`	Monitor- and step-level execution results (HTTP, browser, multi-protocol)
`dt.synthetic.detailed_events`	Request-level detail for HTTP steps (`event.type == "http_step_execution_details"`)

Records are distinguished by `event.type` :

`event.type`	Monitor	Granularity
`http_monitor_execution`	HTTP	One per execution (carries `result.state`)
`http_step_execution`	HTTP	One per request/step
`browser_monitor_execution`	Browser	One per execution
`browser_monitor_step_execution`	Browser	One per clickpath step
`multiprotocol_monitor_execution`	Network (ICMP/DNS/TCP)	One per execution

Path 2 — Metrics (`timeseries`)

Pre-aggregated series for dashboards and trends — already scaled to milliseconds / percent.

Metric key	Description
`dt.synthetic.http.availability` / `dt.synthetic.browser.availability`	Availability rate (%)
`dt.synthetic.http.duration` / `dt.synthetic.browser.duration`	Total duration (ms)
`dt.synthetic.http.dns` , `.tcp_connect_time` , `.tls_handshake_time` , `.time_to_first_byte`	HTTP timing breakdown (ms)
`dt.synthetic.multi_protocol.executions`	Network monitor execution count

Key fields on `dt.synthetic.events`

Field	Type	Description
`dt.synthetic.monitor.id`	string	Monitor ID
`monitor.name`	string	Monitor display name
`step.name`	string	Step display name
`dt.entity.http_check`	string	HTTP monitor entity
`dt.entity.synthetic_location`	string	Location entity (resolve with `entityName()`)
`result.state`	string	`SUCCESS` / `FAIL` (on monitor-execution events)
`result.status.message`	string	e.g. `HEALTHY` , `UNKNOWN_HOST`
`result.status.code`	long	`0` = success; non-zero = failure
`result.statistics.duration`	duration	Total execution time

Field	Type	Description
`result.statistics.response_status_code`	long	HTTP response code

Path 3 — Entity definitions (classic tables and their Smartscape equivalents)

Entity tables (`fetch dt.entity.synthetic_test` , `fetch dt.entity.http_check` , `fetch dt.entity.synthetic_location`) hold monitor and location *definitions* — use them to enumerate configured monitors and resolve names. They still exist and still return rows, but DQL queries against them are now documented as deprecated: *"Queries against classic synthetic entities in DQL are deprecated. They remain supported for as long as Dynatrace Classic is supported, so you can adopt the new model at your own pace."*

Each of them now also has a **Smartscape node type**, and that is the preferred surface. SaaS 1.344 (released 07/27/2026, with a **staged tenant rollout from 07/29/2026**) moved the Digital Experience apps — Synthetic among them — so their queries and filters "primarily use `dt.smartscape.*` entities, instead of `dt.entity.*` ". Verify 1.344 has reached your tenant before relying on the app-side change; the node types themselves are already queryable via DQL regardless, and cost nothing to read (Smartscape queries scan 0 bytes).

Classic entity	Smartscape node type	Smartscape model
`dt.entity.synthetic_test` (browser monitors)	`BROWSER_MONITOR` **plus** `BROWSER_MONITOR_STEP`	`dt.smartscape.browser_monitor` / `..._step`
`dt.entity.http_check`	`HTTP_MONITOR` **plus** `HTTP_MONITOR_STEP`	`dt.smartscape.http_monitor` / `..._step`
`dt.entity.multiprotocol_monitor`	`NETWORK_AVAILABILITY_MONITOR`	`dt.smartscape.network_availability_monitor`
`dt.entity.synthetic_location`	`SYNTHETIC_LOCATION`	`dt.smartscape.synthetic_location`

Two rows in that table cannot be guessed from the classic name, and both fail silently if you guess wrong:

- `multiprotocol_monitor` **was renamed, not transliterated.** The node type is `NETWORK_AVAILABILITY_MONITOR` . `smartscapeNodes "MULTIPROTOCOL_MONITOR"` does **not** raise an error — it returns zero rows, which reads exactly like "this environment has no network monitors".
- `synthetic_test` **splits into two node types.** Clickpath steps are their own nodes (`BROWSER_MONITOR_STEP`), joined to the monitor by a `contains` edge (or `belongs_to` from the step's side) — they are not attributes of the monitor record. Any query that treated `step.name` as a field of the test needs a `traverse` on Smartscape. The same applies to `HTTP_MONITOR_STEP` .

Two smaller conveniences: the node's `name` **field is already the display name**, so the `entityName()` resolution step that classic entity queries needed disappears; and every node carries `id_classic` (`SYNTHETIC_TEST-*` , `HTTP_CHECK-*` , `SYNTHETIC_LOCATION-*`), which is the join key between migrated and unmigrated queries during the transition.

> **Migration mechanics** — for the general rules on classifying a query before migrating it (dimension-first for mass data, `smartscapeNodes` only for entity-list queries), the type and construct mapping, `traverse` , and the `id_classic` bridge, see **FAQ-16: Migrating Classic Entity Selectors to Smartscape**.
>
> **Sources:** Dynatrace SaaS release notes 1.344 (DT docs) — "All latest Dynatrace queries and filters in the above apps now primarily use `dt.smartscape.*` entities, instead of `dt.entity.*` and `dt.rum.application.*` "; Synthetic monitors in Smartscape (DT docs) — the deprecation quoted above. **Dictionary:** `dt.smartscape.browser_monitor` (`BROWSER_MONITOR` ← `dt.entity.synthetic_test`), `dt.smartscape.http_monitor` (`HTTP_MONITOR` ← `dt.entity.http_check`), `dt.smartscape.network_availability_monitor` (`NETWORK_AVAILABILITY_MONITOR` ← `dt.entity.multiprotocol_monitor`), `dt.smartscape.synthetic_location` (`SYNTHETIC_LOCATION` ← `dt.entity.synthetic_location`), plus `dt.smartscape.browser_monitor_step` / `dt.smartscape.http_monitor_step` , read from `dt.semantic_dictionary.models` (`data_object == "smartscape.nodes"` , `classic_models` expanded) 09/28/2026.

> Observed on a live tenant 09/28/2026: `smartscapeNodes "MULTIPROTOCOL_MONITOR"` returned 0 rows with no error, while `smartscapeNodes "NETWORK_AVAILABILITY_MONITOR"` returned 4 — the two "cannot be guessed" traps are observed consequences of that mapping, not documented warnings.

Access scoping: security context on monitors

Synthetic access in Latest Dynatrace is scoped by **security context** values set on monitors — the role management zones played in Classic. *"A monitor supports a maximum of 10 security context values."* IAM policies then condition Synthetic permissions on those values, and the data permissions on the same values:

```
ALLOW synthetic:monitors:read, synthetic:monitors:write, synthetic:monitors:execute WHERE
synthetic:dt.security_context IN ("team-a");
ALLOW storage:metrics:read, storage:events:read, storage:user.events:read, storage:smartscape:read WHERE
storage:dt.security_context MATCH ("team-a");
ALLOW storage:buckets:read WHERE storage:table-name IN ("metrics", "events", "user.events");
```

Four things to know:

- **Classic permissions are not scoped by it.** *"Classic permissions remain functional, but they always apply tenant-wide. You can't scope classic permissions to a subset of monitors, and security contexts don't affect users who have only classic permissions."*
- **Table reads need the bucket permission.** `storage:buckets:read` is *"Required additionally to a table permission"* (IAM policy reference (DT docs)), hence the third statement.
- **Private locations need ActiveGate 1.331+** for execution data to carry the monitor's security context (SYNTH-04 § 3).
- **`dt.security_context` is an array.** `dt.security_context == "team-a"` matches nothing; use `in("team-a", dt.security_context)` (next cell). The same applies in IAM — Permissions in Grail (DT docs): *"Using* `=`, `STARTSWITH` *or* `IN` *when the field holds an array will always return* `false`*."*, so the storage statement above uses `MATCH`. On the validation tenant (10/13/2026) every synthetic event that carried a security context held it as an array. On the validation tenant (10/02/2026) the `==` form matched 0 of 9,311 executions and the `in()` form matched 4,703.

SaaS 1.343 migrated each monitor's management zones to security-context values once; later MZ changes are not synchronized (SYNTH-04 § 3).

> **Sources:** Access control (DT docs) — *"A monitor supports a maximum of 10 security context values."* and *"Classic permissions remain functional, but they always apply tenant-wide."*

```
// Executions for one security context — dt.security_context is an array, so use in(), not ==
// Replace "team-a" with one of your values; the second query shows which values exist.
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")  // one result per
scheduled run
| filter in("team-a", dt.security_context)
| summarize {executions = count(), failed = countIf(result.state == "FAIL")}, by:{monitor.name,
dt.synthetic.monitor.id}

// Which security-context values your executions carry:
// fetch dt.synthetic.events, from: now() - 24h
// | filter endsWith(event.type, "_monitor_execution")
// | summarize {n = count()}, by:{dt.security_context}
```

5. Your First Synthetic Query

Let's explore synthetic monitoring data in your environment.

Count each scheduled run once. A browser monitor that retries on error writes the failed attempt *and* a re-run (`execution.type` `RE_RUN`) for the same scheduled run. The Synthetic events model describes `execution.retry_on_error` as *"Should we ignore this execution attempt (which was a failure) - will there be retry."* Counting both inflates failures: on the validation tenant, 2,015 of 4,701 browser executions in 24 hours were re-runs. The aggregating queries in this notebook drop the superseded attempt with `filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")`. The `coalesce` matters: HTTP executions carry no value for the field, and without it their failures would be filtered out as well.

Monitor names are not unique. A browser monitor and an HTTP monitor can share a name, so per-monitor queries group by `dt.synthetic.monitor.id` as well as `monitor.name`.

Sources: Synthetic events model (DT docs) — *"execution.type MUST be one of the following"* (the list includes `re-run` and `standard`; the tenant returned `RE_RUN` / `STANDARD` for browser and `standard` for HTTP, read 10/02/2026).

```
// Discover configured synthetic monitors in your environment
// PREFERRED -- Smartscape. Browser monitors are BROWSER_MONITOR; the node's `name`
// field IS the display name, so no entityName() resolution is needed.
smartscapeNodes "BROWSER_MONITOR"
| fields name, id, id_classic, url, enabled, frequency
| sort name asc
| limit 50

// Other monitor types:
//   HTTP monitors                 -&gt; smartscapeNodes "HTTP_MONITOR"
//   Network availability monitors  -&gt; smartscapeNodes "NETWORK_AVAILABILITY_MONITOR"
// NOT "MULTIPROTOCOL_MONITOR" -- that node type does not exist and returns zero rows
// rather than an error, which is indistinguishable from having no network monitors.

// FALLBACK (classic surface -- deprecated in DQL, supported for as long as
// Dynatrace Classic is supported):
// fetch dt.entity.synthetic_test
// | fields id, entity.name
// | sort entity.name asc
// | limit 50
```

```
// Count executions by monitor type (from synthetic events)
// event.type distinguishes HTTP / browser / multi-protocol executions.
// This counts every execution, re-runs included (see the note above), which is what you want for volume.
fetch dt.synthetic.events, from: now() - 24h
| summarize count = count(), by: {event.type}
| sort count desc
```

```
// List available synthetic locations
// PREFERRED -- Smartscape. location_type, stage, cloud.provider and geo.* are all
// available on the SYNTHETIC_LOCATION node. The classic entity genuinely has none of
// them, which is why the old guidance said these fields "are not available".
smartscapeNodes "SYNTHETIC_LOCATION", from: now() - 7d  // the default 2 h window hides locations that
reported earlier
| fields name, location_type, stage, cloud.provider, geo.country.name, geo.city.name
| sort name asc
| limit 50

// See SYNTH-04 for the private-vs-public breakdown and a verification caveat on the
// exact location_type literals your tenant returns.

// FALLBACK (classic surface -- deprecated in DQL, and name-only: cloudPlatform, city
// and countryCode really are absent from dt.entity.synthetic_location):
// fetch dt.entity.synthetic_location
// | fields id, entity.name
// | sort entity.name asc
// | limit 50
```

```
// Recent synthetic execution results (last 24 hours)
// Smartscape fields: dt.entity.synthetic_location / dt.entity.synthetic_test on synthetic data are
// deprecated ("will be removed in the future") in favor of dt.smartscape.* — Synthetic events model,
09/21/2026.
// Monitor-level events carry result.state; one row per execution
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")  // one result per
scheduled run
| fields timestamp,
         monitor = monitor.name,
         type = event.type,
         location = getNodeName(dt.smartscape.synthetic_location),
         state = result.state,
         status = result.status.message,
         response_ms = result.statistics.duration / 1ms
| sort timestamp desc
| limit 100
```

```
// Synthetic availability summary (last 24 hours)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")  // one result per
scheduled run
| summarize {
    total_executions = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
  }
| fieldsAdd availability_pct = round((successful * 100.0) / total_executions, decimals: 2)
```

```
// Response time by monitor (last 24 hours)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")  // one result per
scheduled run
| filter isNotNull(result.statistics.duration)
| summarize {
    avg_response_ms = avg(result.statistics.duration / 1ms),
    max_response_ms = max(result.statistics.duration / 1ms),
    executions = count()
  }, by: {monitor.name, dt.synthetic.monitor.id}
| fieldsAdd avg_response_ms = round(avg_response_ms, decimals: 1)
| fieldsAdd max_response_ms = round(max_response_ms, decimals: 1)
| sort avg_response_ms desc
| limit 20
```

Summary

In this notebook, you learned:

✅ **What Synthetic Monitoring is** and how it differs from RUM
✅ **Monitor types** - Browser (single-URL, clickpath), HTTP, and network availability monitors (private locations only)
✅ **Access scoping** - security context on monitors, and the array trap
✅ **Key concepts** - Locations, frequency, outage detection
✅ **Grail data model** - `fetch dt.synthetic.events` for execution detail and `timeseries dt.synthetic.*` for metrics
✅ **Basic DQL queries** - Discover monitors, locations, and results with `result.state` / `result.statistics.*`

Next Steps

Continue to **SYNTH-02: Browser Monitors** to learn how to create and optimize browser-based synthetic tests.

References

- Synthetic on Grail (DT docs)
- Synthetic app (DT docs)
- Synthetic on Grail metrics (DT docs)
- Browser monitors (DT docs)
- Create and configure an HTTP monitor (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.